

Reranking Using Cross Encoders for Precision Retrieval

TECHNICAL REFERENCE GUIDE & ARCHITECTURE OVERVIEW

In **Retrieval-Augmented Generation (RAG)**, the first retrieval step usually brings the **top-k relevant documents** using vector similarity databases such as FAISS, ChromaDB, Pinecone, and others.

But vector search is sometimes *approximate*, meaning:

- Irrelevant chunks may appear in top results
- Important chunks may rank lower
- Semantic similarity alone may miss context precision

To improve accuracy, we use **Reranking**.

What is Reranking?

Reranking is a multi-stage optimization process that means:

- Retrieve candidate documents/chunks using embeddings.
- Re-evaluate those results using a stronger, more context-aware model.
- Sort them again based on actual relevance.

The reranker effectively acts like a **second-level filter** to maximize retrieval quality.

Why Cross Encoders?

There are two main retrieval models utilized in modern information system architectures:

Model	Works How	Speed	Accuracy
Bi-Encoder	Encodes query and document separately	Fast	Medium
Cross-Encoder	Encodes query + document together	Slower	Very High

Bi-Encoder (Vector Search)

Embedding models function by mapping textual units to vectors separately:

- **query** → **vector**
- **document** → **vector**

Then, mathematical metrics such as **cosine similarity** are applied to determine proximity.

Cross Encoder

A Cross Encoder takes the **[Query] + [Document]** together as a unified input, then directly predicts a unified **relevance score**. This gives significantly better precision because the model can perform cross-attention between every token in the query and every token in the document simultaneously.

Architecture

Step 1 — Initial Retrieval

User Query

↓ Vector DB Search

↓ Top 10 Chunks

Step 2 — Reranking

Cross Encoder (query + each chunk)

↓ New relevance scores

↓ Top 3 most relevant chunks

Why Cross Encoders are Better

Because the query and the document are processed **together** rather than independently.

COMPARATIVE RANKING EXAMPLE

Query: *How to reset forgotten password?*

Chunk A: Reset your account password using email verification.

Chunk B: Password policy requires 8 characters.

An standard embedding search may rank both chunks similarly due to the shared keyword "password". However, the Cross Encoder understands that **Chunk A answers the question**, whereas **Chunk B only discusses password rules**. Consequently, Chunk A gets a significantly higher score.

Metadata Filtering: Scoping Data Based on Attributes

In RAG and Vector Databases, **Metadata Filtering** means retrieving documents/chunks only from specific categories, users, dates, departments, file types, etc. Instead of searching the entire vector database, we first apply filters using metadata attributes.

What is Metadata?

Metadata refers to additional information or attributes bound to a document.

Example document: *"Python is used in AI applications."*

Its metadata may be structured as follows:

```
{
  "topic": "AI",
  "author": "Biju",
  "year": 2025,
  "department": "Engineering",
  "source": "training_pdf"
}
```

Why Metadata Filtering?

Without filtering: The Vector DB searches ALL chunks across the entire index, which may retrieve irrelevant departments, files, or users.

With filtering: The system constraints searches exclusively inside the relevant scope. This dramatically improves:

- Precision
- Security
- Speed
- Multi-user isolation

Real-World Example

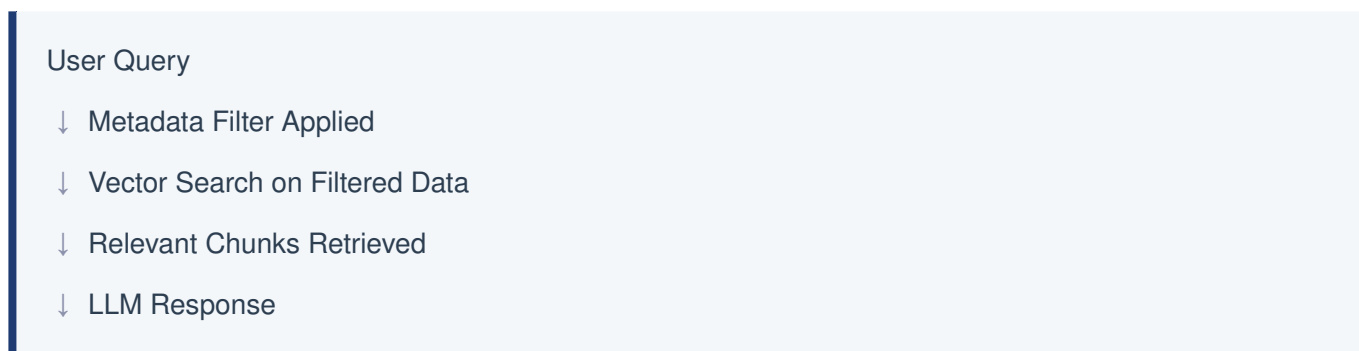
Suppose a company has distinct documents segregated by organizational departments:

Department
HR
Finance
Engineering
Legal

User asks: *"What is leave policy?"*

- **Without metadata filter:** Engineering or operational documents containing the word "leave" may erroneously appear.
- **With filter:** By explicitly specifying **department = "HR"**, only relevant human resource documents are targeted and searched.

RAG Architecture with Metadata Filtering



Example Use Cases

1. Multi-User Chatbot

Each user should access only their own private documents.

```
Filter: {"user_id": 101}
```

2. Time-Based Search

Retrieve only the latest contextual information.

Filter: {"year": 2025}